HTML + Web servers

Basics of HTML

Introduction to HTML

- HTML stands for HyperText Markup Language.
- It is the standard markup language for creating web pages.
- HTML defines the structure and presentation of content on the World Wide Web.
- It consists of a series of tags that define elements on a web page.

History and Evolution of HTML

- HTML was first created by Tim Berners-Lee in 1990.
- The initial version, HTML 1.0, provided a simple way to structure documents.
- HTML 2.0, released in 1995, introduced new features and standardized the
- HTML 3.2, released in 1997, brought improved formatting options and tables.
- HTML 4.0, released in 1997, introduced frames, style sheets, and scripting.
- HTML5, released in 2014, is the latest major version and introduced many new

Importance of HTML in Web Development

- HTML is the foundation of the web and is essential for creating web pages.
- It provides a standard structure that allows web browsers to interpret and display content correctly.
- HTML allows developers to create a variety of elements such as headings, paragraphs, images, links, forms, and more.
- It enables the inclusion of multimedia content, such as videos and audio files.
- HTML works in conjunction with other technologies like CSS and JavaScript to enhance the functionality and appearance of web pages.
- HTML is supported by all major web browsers, ensuring cross-platform compatibility.
- It is a relatively easy language to learn and use, making it accessible to both beginners and experienced developers.

Importance of HTML in Web Development

- HTML is crucial for search engine optimization (SEO) as it provides the structure and context for search engines to understand web page content.
- With the increasing popularity of mobile devices, HTML plays a vital role in creating responsive web design that adapts to different screen sizes.
- HTML is constantly evolving, with new features and improvements being introduced regularly.

Basic HTML Document Structure

An HTML document consists of a combination of HTML tags, elements, and attributes.

The basic structure of an HTML document includes the following components:

a. <!DOCTYPE>: Specifies the HTML version and document type.
b. <html>: Represents the root element of an HTML page.
c. <head>: Contains metadata and other non-visible elements.
d. <title>: Defines the title of the HTML document, displayed in the browser's title bar.
e. <body>: Contains the visible content of the web page.

HTML Tags, Elements, and Attributes

Tags: HTML tags are used to mark up and define elements within an HTML document. They are enclosed in angle brackets (< and >).

Elements: An HTML element consists of an opening tag, content, and a closing tag. The opening tag defines the beginning of the element, and the closing tag denotes the end.

Attributes: HTML attributes provide additional information about an element. They are specified within the opening tag and are written as name-value pairs.

HTML Tags, Elements, and Attributes

Example of a simple HTML element:

- Opening tag:
- Content: This is a paragraph element.
- Closing tag:
- Result: This is a paragraph element.

DOCTYPE

- DOCTYPE stands for Document Type Declaration.
- It is an instruction that tells the web browser how to interpret an HTML document.
- DOCTYPE declaration is placed at the beginning of an HTML document.
- It specifies the HTML version being used and provides information about the document type.
- The DOCTYPE declaration is crucial for ensuring cross-browser compatibility and proper rendering of HTML elements.
- The DOCTYPE declaration helps the web browser understand which rules and standards to apply when rendering the HTML document.
- It ensures that the web page is displayed correctly and consistently across different browsers and devices.

DOCTYPE

- Using the appropriate DOCTYPE declaration is essential for compatibility with HTML validators and accessibility tools.
- The DOCTYPE declaration also plays a role in search engine optimization (SEO) by helping search engines interpret the content accurately.

Header Tags (h1-h6)

- HTML provides six levels of header tags: h1, h2, h3, h4, h5, and h6.
- Header tags are used to define headings and subheadings within an HTML document.
- The h1 tag represents the highest level of heading, while h6 represents the lowest level.
- Usage: <h1> to <h6>
- Example: <h1>This is a Heading</h1>

Paragraph, Line Break, and Horizontal Rule Tags

- : The paragraph tag is used to define a paragraph of text.
-
: The line break tag is used to insert a line break within a paragraph or text.
- <hr>: The horizontal rule tag is used to create a horizontal line to visually separate content.

- Usage: for paragraphs,
 for line breaks, <hr> for horizontal rules.
- Example: This is a paragraph.
This is a line break.<hr>

Link and Image Tags

- <a>: The link tag is used to create hyperlinks, allowing users to navigate between web pages.
- Usage: Link Text
- Example: Visit Example

- <img>: The image tag is used to insert images into an HTML document.
- Usage: <img src="image_url" alt="alternative_text">
- The src attribute specifies the URL of the image file.
- The alt attribute provides alternative text that is displayed if the image cannot be loaded.
- Example: <img src="image.jpg" alt="Description of the image">

- Images can also be hyperlinked by wrapping the <img> tag within an <a> tag.
- Example: <img src="image.jpg" alt="Description">

Unordered and Ordered Lists

- HTML provides two types of lists: unordered lists and ordered lists.
- Unordered lists are used to create bullet-pointed lists.
- Ordered lists are used to create numbered or lettered lists.
- Usage: <ul> for unordered lists, <ol> for ordered lists.

Example:

Unordered list	Ordered list
  - Item 1 - Item 2 - Item 3 	 - First item - Second item - Third item

Description Lists

- Description lists are used to create lists with terms and their corresponding descriptions.
- Usage: <dl> for the description list, <dt> for the term, and <dd> for the description.

Example:

```
<dl>
  <dt>Term 1</dt>
  <dd>Description 1</dd>
  <dt>Term 2</dt>
  <dd>Description 2</dd>
  <dt>Term 3</dt>
  <dd>Description 3</dd>
```

Tables, Rows, Columns, and Headers

- Tables are used to organize data into rows and columns.
- The structure of a table includes table rows (<tr>), table data cells (<td>), and table headers (<th>).
- <td> is used for regular data cells, while <th> is used for header cells.
- Usage:
    a. <table> for the table container.
    b. <tr> for table rows.
    c. <td> for regular data cells.
    d. <th> for header cells.

Tables, Rows, Columns, and Headers

Example (tables):

```
<table>
  <tr>
    <th>Header 1</th>
    <th>Header 2</th>
  </tr>
  <tr>
    <td>Data 1</td>
    <td>Data 2</td>
  </tr>
  <tr>
    <td>Data 3</td>
    <td>Data 4</td>
  </tr>
</table>
```

Form Tag and its Attributes

- The <form> tag is used to create a form in HTML.
- Forms are used to collect user input and send it to a server for processing.
- The <form> tag has various attributes that control its behavior and appearance.
- Commonly used form attributes include:
    a. action: Specifies the URL or server-side script that will process the form data.
    b. method: Specifies the HTTP method to be used when submitting the form (e.g., GET or POST).
    c. enctype: Specifies the encoding type for the form data when submitting (e.g., "application/x-www-form-urlencoded" or "multipart/form-data").

Input Types

- The <input> tag is used to create an input field within a form.
- The type attribute determines the type of input field.
- Example: <input type="text" name="username" placeholder="Enter your username">

- Radio buttons and checkboxes allow users to select one or multiple options, respectively.
- Radio buttons:
    a. Example: <input type="radio" name="gender" value="male"> Male
    b. Example: <input type="radio" name="gender" value="female"> Female

Input Types

- Checkboxes:
  a. Example: <input type="checkbox" name="interest" value="music"> Music
  b. Example: <input type="checkbox" name="interest" value="sports"> Sports

- The submit input type is used to create a submit button within a form.
- When clicked, it submits the form data to the specified server-side script.
- Example: <input type="submit" value="Submit">

Labels, Fieldset, and Legends

- Labels are used to provide a text description for form elements.
- They improve form accessibility and usability.
- Example: <label for="username">Username:</label><input type="text" id="username" name="username">
- Fieldset and legends are used to group related form elements and provide a title for the group.
- Example:

  <fieldset>

  <legend>Personal Information</legend>

  <!-- Form elements go here →

  </fieldset>

Apache Web Server

Introduction to Apache Web Server

The Apache HTTP Server, commonly known as Apache, is a popular open-source web server software. It is developed and maintained by the Apache Software Foundation.

- Apache is widely used to serve static and dynamic web content on the internet.
- Apache has become the most widely used web server software in the world.
- It is known for its stability, security, and extensibility.
- Apache supports multiple platforms, including Linux, Unix, Windows, and macOS.
- It is highly customizable and can be integrated with various programming languages, modules, and frameworks.
- Apache's modular architecture allows administrators to enable or disable specific features based on their needs.

Apache Web Server history

- Apache originated from the National Center for Supercomputing Applications (NCSA) in the mid-1990s.
- The initial version, called "NCSA HTTPd," was developed by Rob McCool.
- In 1995, a group of developers started the Apache Group and released the Apache HTTP Server based on the NCSA code.
- Apache quickly gained popularity due to its stability, performance, and open-source nature.

Apache Web Server key features

Some key features of Apache include:

a. HTTP/1.1 compliance: Supports the latest HTTP protocol standards.
b. Virtual hosting: Allows hosting multiple websites on a single server.
c. Security: Provides various security modules and features, including SSL/TLS encryption.
d. Performance: Optimized for high performance and scalability.
e. Extensibility: Offers a wide range of modules and APIs for customization.

Apache Web Server Architecture

Apache follows a modular architecture that allows it to handle concurrent connections and serve web content efficiently. The core components of Apache's architecture are the Multi-processing Modules (MPMs):

- MPMs are responsible for managing the creation and handling of Apache processes/threads to handle client requests.
- Different MPMs are available in Apache, allowing administrators to choose the most suitable one based on their needs.

Worker MPM & Prefork

Worker MPM:

- Worker MPM is a thread-based MPM available in Apache.
- It uses multiple threads within each process to handle client requests concurrently.
- Each thread can serve multiple requests simultaneously, leading to efficient resource utilization.

Prefork:

- Prefork MPM is a process-based MPM available in Apache.
- It creates multiple independent processes, each capable of handling one client request at a time.
- Prefork MPM provides increased stability and isolation but can consume more resources compared to Worker MPM.

Event MPM

- Event MPM is a hybrid MPM introduced in Apache 2.4.
- It combines the benefits of both the Worker and Prefork MPMs.
- Event MPM uses a pool of threads to handle client requests and separates the handling of connections and request processing, resulting in improved performance and resource efficiency.

Process-based vs. Thread-based Architecture

In a process-based architecture, each process handles one client request at a time.	In a thread-based architecture, multiple threads within a process handle concurrent client requests.
Process-based architectures provide better isolation and stability but may consume more resources due to the overhead of process creation.	Thread-based architectures offer better resource utilization and scalability but require careful handling to ensure thread safety.

Apache configuration

- Apache configuration involves setting up various parameters and options to customize the behavior of the web server.
- The main configuration file for Apache is httpd.conf.

httpd.conf File

- The httpd.conf file is the primary configuration file for Apache.
- It contains global settings that apply to the entire web server.
- The location of the httpd.conf file may vary depending on the Linux distribution, but it is typically found in the Apache configuration directory.

Apache configuration

Directives

- Directives are the basic building blocks of Apache configuration.
- They specify various settings and behaviors of the web server.
- Directives are defined within the httpd.conf file using a specific syntax.

Sections and Containers

- Sections are used to group related directives and define their scope.
- They provide a way to organize and manage different aspects of the server's behavior.
- Common sections in the httpd.conf file include the main server section, virtual host sections, and directory sections.

Apache configuration

.htaccess Files

- .htaccess files are per-directory configuration files.
- They allow local configurations within a specific directory or its subdirectories.
- .htaccess files provide a way to override certain Apache settings on a per-directory basis.

.htaccess Examples

- Some common use cases for .htaccess files include:
  - Setting directory-specific access controls.
  - Enabling or disabling certain features.
  - Redirecting URLs.
  - Customizing error pages.

Apache configuration

.htaccess Best Practices

- It is important to use .htaccess files judiciously, as they can impact server performance.
- Frequent use of .htaccess files can result in additional file system operations and slowdowns.
- It is recommended to use .htaccess files only when necessary and to place commonly used directives in the main httpd.conf file for better performance.

Apache Modules

- Apache modules are software components that extend the functionality of the Apache web server.
- Modules can add features, improve performance, enhance security, and enable integration with other technologies.

Core Modules

- Apache comes with a set of core modules that are essential for the basic functioning of the web server.
- Core modules provide functionality such as handling HTTP requests, logging, authentication, and access control.
- Some examples of core modules include mod_rewrite, mod_ssl, and mod_auth_basic.

Apache Modules

Extra and Third-party Modules

- Extra modules are additional modules that are distributed with Apache but not considered core.
- They provide additional functionality that may not be needed by all installations.
- Third-party modules are developed by the Apache community or external developers and are not distributed with Apache by default.

Some popular extra modules include:

a. mod_proxy: Enables reverse proxying and load balancing.
b. mod_php: Integrates PHP scripting language with Apache.
c. mod_deflate: Provides gzip compression for reducing the size of transmitted data.
d. mod_security: Enhances web server security by applying various security rules.

Apache Modules Activation & Configuration

Activation

- To activate a module, it must be loaded into the Apache configuration.
- Apache provides the LoadModule directive to load modules dynamically.
- The directive specifies the module name and the path to the module's shared object file (.so).

Module Configuration

- Once a module is activated, its behavior can be customized using module-specific configuration directives.
- Module configuration directives are specified in the Apache configuration files.
- Different modules have different configuration options, and they can be used to fine-tune the behavior of the module.

Apache Modules Configuration

Some examples of module-specific configuration directives include:

- mod_rewrite: RewriteRule, RewriteCond for URL rewriting.
- mod_ssl: SSLCertificateFile, SSLCipherSuite for configuring SSL/TLS.
- mod_proxy: ProxyPass, ProxyPassReverse for reverse proxying.

Apache Management Tools

- Apache management involves various tasks to control and monitor the Apache web server.
- These tasks include starting, stopping, and restarting the server, managing logs, and obtaining server status and information.

Apache Management Tools

Starting, Stopping, Restarting Apache

- Apache can be started, stopped, and restarted using command-line tools or service management utilities.
- On most Linux distributions, the command 'sudo systemctl start|stop|restart apache2' is used.
- Other commands like 'apachectl start|stop|restart' or 'service apache2 start|stop|restart' may be used depending on the distribution.

Apache Management Tools

Error Logs and Access Logs

- Apache generates error logs and access logs to record server-related errors and client requests.
- Error logs provide information about issues encountered during server operation.
- Access logs capture details of client requests, including IP addresses, requested files, and HTTP status codes.

Apache Management Tools

Error Log Location

- The location of the error log file may vary depending on the Linux distribution and Apache configuration.
- Common locations include '/var/log/apache2/error.log' or '/var/log/httpd/error_log'.
- Error log files can be viewed using text editors or command-line tools like 'tail' or 'less'.

Apache Management Tools

Access Log Location

- The location of the access log file also depends on the Linux distribution and Apache configuration.
- Common locations include '/var/log/apache2/access.log' or '/var/log/httpd/access_log'.
- Access log files can be analyzed using various log analysis tools or viewed using text editors.

Apache Management Tools

Server Status and Info

- Apache provides built-in features to obtain server status and information.
- The mod_status module can be enabled to access a status page showing details about current connections, requests, and server performance.
- The server-info page provides comprehensive information about the Apache server configuration.

Apache Management Tools

Server Status Page

- The server status page provides real-time information about server performance and connections.
- It can be accessed by visiting the server's URL followed by '/server-status', e.g., 'http://localhost/server-status'.
- The status page displays details such as current requests, CPU usage, memory usage, and worker/thread information.

Apache Management Tools

Server Info Page

- The server info page provides detailed information about the Apache server configuration.
- It can be accessed by visiting the server's URL followed by '/server-info', e.g., 'http://localhost/server-info'.
- The info page shows server version, loaded modules, server variables, and more.

Apache Virtual Hosts

- Apache Virtual Hosts allow hosting multiple websites on a single server.
- Each virtual host has its own configuration and can have a unique domain name or IP address.
- Virtual Hosts enable the server to serve different content based on the requested domain name or IP address.
- This allows multiple websites to be hosted on the same physical server.

Apache Virtual Hosts types

IP-based Virtual Hosts

- IP-based Virtual Hosts use different IP addresses assigned to the server to differentiate between websites.
- Each IP address is associated with a specific website, allowing Apache to serve the appropriate content based on the IP address used in the request.

Name-based Virtual Hosts

- Name-based Virtual Hosts use the requested domain name to differentiate between websites.
- Apache matches the domain name specified in the request header with the configured Virtual Hosts and serves the content accordingly.
- Name-based Virtual Hosts are the most common type of Virtual Hosts.

Apache Virtual Host Configuration

- Virtual Hosts are configured in the Apache configuration files.
- Each Virtual Host has its own configuration block, which includes directives specific to that host.
- The main configuration file (httpd.conf) or separate files included from it can be used to define Virtual Hosts.

Examples:

<VirtualHost> directive is used to define a Virtual Host.

The directive includes the IP address or "*" for all addresses, the port number, and the domain name or server name.

Inside the <VirtualHost> block, specific configuration directives for that host can be defined.

Apache Virtual Host Configuration

```
<VirtualHost *:80>
  ServerName www.example.com
  DocumentRoot /var/www/example
  <Directory /var/www/example>
    Options FollowSymLinks
    AllowOverride All
    Require all granted
  </Directory>
</VirtualHost>
```

Apache Virtual Host Configuration Best Practices

- Use descriptive names for Virtual Hosts to easily identify their purpose.
- Ensure that the DNS records for the domain names used in Name-based Virtual Hosts are properly set up.
- Regularly test and verify the configuration of Virtual Hosts to ensure proper functionality.

Introduction to Apache Tomcat

- Apache Tomcat is a lightweight, standalone web server that provides a Java-based environment for executing Java web applications.
- It acts as a container for running Java servlets, JavaServer Pages (JSP), and JavaServer Faces (JSF) applications.

Relation between Apache HTTP Server and Tomcat

- Apache HTTP Server and Apache Tomcat are two separate projects within the Apache Software Foundation.
- Apache HTTP Server is a general-purpose web server primarily used for serving static content.
- Tomcat, on the other hand, is a servlet container and is designed specifically for executing Java-based web applications.

Introduction to Apache Tomcat

How Apache HTTP Server and Tomcat Work Together

- In many deployments, Apache HTTP Server is used as a front-end server to handle client requests and serve static content.
- When a request for a dynamic web application (e.g., servlets or JSP) is received, Apache HTTP Server can pass the request to Tomcat using the Apache Tomcat Connector (mod_jk or mod_proxy).

Introduction to Apache Tomcat

Advantages of Using Apache HTTP Server with Tomcat

- By using Apache HTTP Server in front of Tomcat, you can take advantage of its features, such as load balancing, caching, and SSL/TLS termination.
- Apache HTTP Server can also handle static content more efficiently, offloading the workload from Tomcat.
- This setup provides a scalable and robust architecture for serving dynamic Java-based web applications.

Apache Web Server Installation

Apache Installation Prerequisites

Before installing Apache, ensure that the following prerequisites are met:

- A compatible operating system (Linux, Windows, MacOS)
- Sufficient system resources (CPU, memory, disk space)
- Administrative privileges to install software

Apache Installation. Linux

- Apache can be installed on Linux using package managers like apt, yum, or dnf.
- The package name is usually 'httpd' or 'apache2'.

Example command: 'sudo apt install apache2' (for Debian-based systems)

Apache Installation. Windows

- Apache can be installed on Windows using the official Apache Lounge distribution or third-party installers like XAMPP or WampServer.
- Download the installer package from the official website or third-party sources.
- Follow the installation wizard and choose the desired options.

Apache Installation. MacOS

- Apache is pre-installed on MacOS. However, it may require additional configuration.
- To enable Apache on MacOS, open Terminal and run the command 'sudo apachectl start'.
- Verify the installation by opening a web browser and navigating to 'http://localhost'.

Apache Installation Verification

- After installation, verify the Apache installation by accessing the default Apache web page.
- Open a web browser and enter 'http://localhost' or 'http://<your-server-ip>' in the address bar.
- If Apache is running correctly, you should see the default Apache web page.

Configuring Apache

- Apache can be customized and configured to meet specific requirements.
- Configuration changes are made in the 'httpd.conf' file, which is the main configuration file for Apache.
- The 'httpd.conf' file is located in the 'conf' directory of the Apache installation.
- Use a text editor to open the file (e.g., 'sudo nano /etc/httpd/conf/httpd.conf' on Linux).
- Make necessary configuration changes based on requirements.

Configuring Apache

Common Configuration Changes

- ServerRoot: Specifies the root directory for Apache installation.
- DocumentRoot: Specifies the directory where the website files are stored.
- Port: Configures the port number on which Apache listens for incoming connections.
- DirectoryIndex: Specifies the default file to serve when a directory is requested.

Virtual Host Configuration

- Virtual Hosts can be configured in the 'httpd.conf' file to host multiple websites on a single server.
- Use the <VirtualHost> directive to define each Virtual Host and specify the necessary configuration inside the directive.

Configuring Apache

Testing Configuration Changes

- After making configuration changes, it's important to test the changes to ensure they are working as expected.
- Use the command 'sudo apachectl configtest' on Linux to check the configuration syntax.
- If the test is successful, restart Apache to apply the changes.

Apache Modules

How to Enable/Disable Modules

- Apache modules can be enabled or disabled using the 'a2enmod' and 'a2dismod' commands on Linux.
- Example: 'sudo a2enmod ssl' to enable the SSL module.
- After enabling or disabling a module, Apache needs to be restarted for the changes to take effect.

Apache Virtual Hosts

Creating and Configuring Virtual Hosts

Step 1: Create a new configuration file for the Virtual Host in the 'sites-available' directory (e.g., 'sudo nano /etc/apache2/sites-available/example.com.conf').

Step 2: Configure the Virtual Host by specifying the domain, document root, log files, and other necessary settings.

Step 3: Enable the Virtual Host by creating a symbolic link to the 'sites-enabled' directory (e.g., 'sudo ln -s /etc/apache2/sites-available/example.com.conf /etc/apache2/sites-enabled/').

Step 4: Restart Apache to apply the changes.

Apache Virtual Hosts

Troubleshooting Common Issues

Issue: Virtual Host not working

- Check if the configuration file is in the 'sites-available' directory and correctly symlinked to 'sites-enabled'.
- Verify that the domain name is resolving to the correct IP address.

Issue: Permission denied errors

- Ensure that the document root and other directories have the necessary permissions for Apache to access and serve the files.
- Check the file and directory ownership and permissions using 'ls -l'.

Apache Virtual Hosts

Troubleshooting Common Issues

Issue: Incorrect DNS configuration

- Verify that the domain name is correctly configured in DNS settings and resolves to the correct IP address.

Issue: Conflicting Virtual Hosts

- Check if there are any overlapping or conflicting configurations between Virtual Hosts.
- Ensure that the 'ServerName' and 'ServerAlias' directives are set correctly for each Virtual Host.

Issue: Syntax errors in Virtual Host configuration

- Use the command 'sudo apache2ctl configtest' to check for syntax errors in the Virtual Host configuration files.
- Review the error message and correct any syntax mistakes.

Q&A